Adding AI / LLMs to Our Project

Topic 22: Personalization and Recommendation Under Privacy Constraints — AI/LLM ideas for team discussion

Why we are looking at this

All four of our models use SASRec, a small Transformer published in 2018. It is deep learning, but it does not use any large language model, and the architecture is not new. We want the project to use current AI technology.

Our new dataset, **Amazon Reviews 2023 — Beauty and Personal Care**, makes this practical. Besides ratings, it has text for every product: titles, descriptions and features. An LLM can read that text, which SASRec cannot.

Each idea below either fixes a real problem in our project or adds something to the privacy question we are studying. The baseline stays plain SASRec, so we can measure exactly what the AI adds.

The two problems an LLM can help with

1. Cold start. SASRec learns each item only from who bought it. An item with little or no purchase history gets a poor representation, so it is almost never recommended. On the All Beauty 2018 data we used before switching datasets, 82% of test items never appeared in training. The 2023 data is much larger, but still has a long tail of rarely bought products.

2. On-device cost for Model 3. SASRec stores one row of numbers per item. Our current subset has 25,903 items, so with 50 numbers per item the table is about 5 MB, which already fits on a phone. Size alone is therefore a weak motivation. What matters more on a device is memory and speed once an LLM-based component is added (Idea 3), and cold start, which affects every model.

What is current in recommendation and privacy (2025–2026)

Five trends shape the ideas in this document:

- **Generative recommendation.** Instead of scoring every item, the model *generates* the next item as a short code, the way an LLM generates words. Meta's HSTU (2024) and Kuaishou's OneRec (2025) moved this into large production systems.
- **LLM-based embeddings.** Text embedding models are now built on LLMs (for example, the Qwen3-Embedding family, 2025), and are much stronger than the BERT-sized encoders used a few years ago.
- **Small LLMs on phones.** Models designed for devices, such as Gemma 3n, Llama 3.2 (1B and 3B) and Phi-4-mini, make on-device LLM features realistic.
- **Private LLM training.** Differential privacy now works at LLM scale (Google's VaultGemma, 2025, was trained with DP from scratch), and federated fine-tuning with LoRA adapters lets devices improve a shared model by sending only small updates.
- **LLM-powered privacy attacks.** Modern LLMs are good at inferring personal details

from indirect clues, which makes measuring leakage more important than ever.

Summary of the ideas

Idea	What the AI does	Benefit	Model	Effort
1. LLM item embeddings	Reads product text, turns it into numbers	Fixes cold start	All four	Low–Med
2. Generative recommendation with semantic IDs	Represents items as short codes and generates them	Modern architecture, helps cold start	Model 3	Med–High
3. On-device LLM re-ranker	Small phone-sized LLM re-orders the top items	Privacy vs quality trade-off for LLMs	Model 3	Medium
4. Federated LoRA	Devices fine-tune a small LLM, share only adapters	Modern federated learning	Model 2	High
5. DP for the LLM component	Trains the LLM part with differential privacy	Formal privacy for the AI part	Model 4	High
6. LLM as privacy attacker	Tries to infer private facts about users	Stronger leakage measurement	Evaluation	Medium

Idea 1 — LLM item embeddings (core idea, all four models)

What it is. We pass each product's title, description and features through a modern LLM-based embedding model once. Each product gets a list of numbers that captures what it is. SASRec then starts from these numbers instead of random ones, so two similar products start out close together, even if one has never been bought.

Candidate embedding models: **Qwen3-Embedding** (2025, available in small sizes) or **BLaIR** (2024), which was trained on our own dataset by the lab that released it. Comparing the two is itself a useful small result.

Why it fits. It directly attacks cold start, it works the same way in all four models so the comparison stays fair, and product text is public, so it adds no privacy concern.

How we would measure it. NDCG@10 and Hit@10 for plain SASRec versus text-initialized SASRec, overall and separately for rarely bought items.

Effort. Low to medium. Encoding about a million product texts once is a few GPU hours on an RTX 4090.

Idea 2 — Generative recommendation with semantic IDs (Model 3)

What it is. Each product's embedding from Idea 1 is turned into a short code, for example four numbers each between 0 and 255. The model stores four small tables instead of one row per item, and recommends by generating the next item's code. This is the approach behind TIGER (2023) and the generative recommenders now used in industry, such as OneRec (2025).

Why it fits.

- Similar products share codes, so it helps cold start, including items never bought in training.
- It also shrinks the item table further, from about 5 MB to well under 1 MB. That is useful but modest at our dataset size, so it is a secondary benefit.
- It puts Model 3 on the architecture the field is moving to, rather than only applying standard compression.

How we would measure it. Model size, single-thread CPU latency and NDCG@10 for three versions: full model, standard compression (quantization), and semantic IDs.

Effort. Medium to high. It needs a small extra model (an RQ-VAE) to learn the codes.

Idea 3 — On-device LLM re-ranker (Model 3)

What it is. SASRec produces its top 20 items, and a small LLM that could run on a phone (for example Gemma 3n or Llama 3.2 1B) re-orders them using the product text and the user's recent items. We compare it with a larger cloud-sized LLM doing the same job. The cloud LLM is likely better, but it requires the user's history to leave the device.

Why it fits. It extends our privacy question to LLMs: how much quality do we give up by keeping the LLM on the device? That is exactly the trade-off companies face with on-device AI today.

How we would measure it. NDCG@10 for no re-ranking, on-device LLM and cloud-sized LLM, plus on-device latency and memory.

Effort. Medium. LLM inference is slow, so we would evaluate on a sample of test users.

Idea 4 — Federated LoRA (Model 2 extension)

What it is. LoRA fine-tunes an LLM by training a small adapter instead of the whole model. In the federated version, each simulated device fine-tunes a small LLM on its own user's history and sends only the adapter update, which is a tiny fraction of the model, to be combined with secure aggregation.

Why it fits. It is the current way to do federated learning with LLMs, and it keeps Model 2's core idea: raw data never leaves the device. Communication cost, which Model 2 already measures, is where LoRA helps most.

How we would measure it. NDCG@10, bytes sent per round, and rounds to converge, compared with federated SASRec.

Effort. High. Best treated as an extension once federated SASRec works.

Idea 5 — Differential privacy for the LLM component (Model 4 extension)

What it is. If we fine-tune an LLM on user histories (in Idea 3 or 4), that LLM can memorize them. We apply the same user-level DP-SGD that Model 4 uses to the LLM's adapter. VaultGemma (2025) showed that DP training is practical even for LLMs.

Why it fits. It means the AI component carries the same formal privacy guarantee (ε, δ) as the rest of Model 4, instead of being the weak point.

How we would measure it. NDCG@10 at several ε values, alongside the existing Model 4 sweep.

Effort. High. Only worth doing if Idea 3 or 4 is adopted.

Idea 6 — An LLM as the privacy attacker (evaluation, all four models)

What it is. We give a current open LLM what each model reveals about a user, such as their recommendation list, and ask it to infer private facts: age group, gender, or sensitive concerns like acne or hair loss. We run the same attack against all four models.

Why it fits. It adds a modern, realistic attack next to membership inference, and answers a clear question: do federated, on-device and DP training protect users against an AI attacker?

How we would measure it. Attack success rate for each model, compared with random guessing.

Effort. Medium. An open LLM running in the GPU Lab, tested on a sample of users.

Ethics. Public dataset only, results reported in aggregate, no attempt to identify real people.

Recommended plan

	What it uses	AI component
Baseline	Plain SASRec, published configuration	None, kept as the reference
Model 1 — Centralized	SASRec with LLM item embeddings	Idea 1
Model 2 — Federated	Federated SASRec with LLM item embeddings	Idea 1, Idea 4 as stretch goal

Model 3 — On-device	Semantic-ID generative model, on-device LLM re-ranker	Ideas 1, 2 and 3
Model 4 — DP	DP-SASRec with LLM item embeddings	Idea 1, Idea 5 as stretch goal
Evaluation	Membership inference plus LLM attacker	Idea 6

Core (do these): Ideas 1, 2 and 6. They are realistic for our timeline and address cold start and on-device cost.

Stretch (if time allows in 298B): Ideas 3, 4 and 5.

Is this realistic on our hardware?

Yes, with sensible model sizes. The GPU Lab's RTX 4090 (24 GB) and RTX 5090 (32 GB) can:

- encode a million product texts with a small embedding model in a few hours;
- run 1B to 8B parameter open LLMs for inference, using 4-bit versions for the larger ones;
- fine-tune 1B to 3B models with LoRA.

We should not plan anything that needs full fine-tuning of large models.

What this changes in our plan

- **Scope.** Workbook section 2.2.5 lists product metadata as out of scope and says architecture changes go to the instructor first. We need Professor Shim's approval before committing.
- **Data pipeline.** The ETL needs to keep product text (title, description, features) in a separate table keyed by item, so the interaction data stays identical for all models.
- **Timeline.** Idea 1 can be prototyped in 298A alongside Model 1. Ideas 2 to 6 belong in 298B.

Papers to read

Every title below must be found, read and checked before it is cited. Some recent work may have newer versions; search the 2025 and 2026 proceedings of RecSys, KDD, SIGIR and WWW, and arXiv cs.IR.

Recent (2024–2025)

- **Trend, Idea 2:** Zhai et al. (ICML 2024), *Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations* (HSTU)
- **Idea 2:** Deng et al. (2025, Kuaishou), *OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment*
- **Idea 2:** Singh et al. (RecSys 2024), *Better Generalization with Semantic IDs: A Case Study in Ranking for Recommendations*

- **Idea 1 and the dataset:** Hou et al. (2024), *Bridging Language and Items for Retrieval and Recommendation* (BLaIR)
- **Idea 1:** Qwen3-Embedding technical report (Alibaba, 2025)
- **Idea 5:** VaultGemma technical report (Google, 2025)
- **Idea 6:** Staab et al. (ICLR 2024), *Beyond Memorization: Violating Privacy via Inference with Large Language Models*
- **Idea 3:** Hou et al. (ECIR 2024), *Large Language Models are Zero-Shot Rankers for Recommender Systems*

Foundations

- **Idea 2:** Rajput et al. (NeurIPS 2023), *Recommender Systems with Generative Retrieval* (TIGER)
- **Idea 1:** Hou et al. (KDD 2022), *Towards Universal Sequence Representation Learning for Recommender Systems* (UniSRec)
- **Idea 4:** Hu et al. (ICLR 2022), *LoRA: Low-Rank Adaptation of Large Language Models*